Daily Progress Report

Week 4_Day4: Browser Storage & Modules (ES6)

Program	3-Month MERN Stack Internship Program
Intern Name	Mohsin Ali
Week	Week_4 (Day 4)
Date	16 July, 2026

Day 4: Browser Storage & Modules (ES6)

1.1 Topics Studied

- Browser Web Storage API mechanics and use cases
- Key differences between localStorage (persistent) and sessionStorage (session-based)
- Data Serialization using JSON.stringify()
- Data Deserialization using JSON.parse()
- Client-side data persistence across page reloads
- ES6 Module syntax (import and export statements)
- Code splitting, separation of concerns, and modular JavaScript architecture

1.2 Task Description

Enhanced the existing Task Tracker application to support client-side data persistence and a modern, modular codebase. The primary objective was to utilize the browser's **localStorage** API to save the array of tasks, ensuring the data persists across page reloads by saving and retrieving parsed JSON data. Additionally, the project focused on refactoring the application architecture using ES6 modules (**import** and **export**) to split the JavaScript into multiple manageable files, separating storage logic, UI rendering, and core functionality.

1.3 Implementation Details

- Refactored the previously monolithic JavaScript code into dedicated ES6 modules, separating core application logic from DOM manipulation and storage operations.

- Utilized the export keyword to share specific functions and variables across files, and the import keyword to bundle them into the main entry script.
- Implemented localStorage.setItem() coupled with JSON.stringify() to automatically serialize and save the working array of task objects whenever a task is added, completed, or deleted.
- Added initialization logic utilizing localStorage.getItem() to detect and retrieve existing saved tasks the moment the application loads.
- Used JSON.parse() to deserialize the retrieved JSON string back into a functional JavaScript array.
- Developed a UI repopulation routine that iterates over the parsed local storage data and dynamically renders the preserved tasks back onto the DOM upon application startup.
- Ensured tight synchronization between the modularized JavaScript array and localStorage to guarantee zero data loss during active user sessions.

1.4 Outcome

The Task Tracker application was successfully upgraded with persistent state management and a clean, modular architecture. Implementing ES6 modules significantly improved code maintainability and readability by establishing a clear separation of concerns. Furthermore, bridging the gap between dynamic UI rendering and **localStorage** successfully eliminated data loss upon page reloads. The application now functions as a resilient, state-aware tool, strengthening my overall grasp of modern JavaScript project structures and client-side storage.

Screenshots:

Screenshots: